Towards Automating the Life Cycle Management of Digital Twins

Gwendal Beaumont[1,2][0000−0001−9861−4008], Antoine Beugnard[1][0000−0002−3096−237X], Salvador Martínez[1][0000−0002−3978−9876], Christelle Urtado[2][0000−0002−6711−8455], and Sylvain Vauttier[2][0000−0002−5812−1230]

[1] IMT Atlantique, Lab-STICC, UMR 6285, Brest, France
{gwendal.beaumont,antoine.beugnard,salvador.martinez}@imt-atlantique.fr
[2] EuroMov Digital Health in Motion, Univ. Montpellier & IMT Mines Ales, Ales, France
{christelle.urtado,sylvain.vauttier}@mines-ales.fr

Abstract. Digital Twins (DTs) are key to Industry 4.0 and IoT systems. By definition, as virtual counterparts of a Physical Object (PO), they have to be kept up-to-date with the PO at a fast pace, leading to multiple reconfigurations, reconnections and redeployments throughout their lifetime. On the one hand, there is a growing interest around DT platforms and DT modeling languages from both academia and industry. However, systematic operational methodologies and practical tools for managing DTs resources and life cycle are more scarce. This paper identifies the need for a domain-agnostic solution to systematically manage the DT life cycle. On the other hand, *GitOps* provides principles and concrete actionable tools, such as CI/CD pipelines, declarative cluster state and version control, to manage software architectures that needs to: (1) be redeployed swiftly without interruptions and, (2) be versioned. By combining the static Digital Twin Definition Language and the GitOps framework, a new way of *automating life cycle management of DTs* at runtime is proposed in this paper. A conceptualization of core DT life cycle operations at runtime such as `connect`, `synchronize`, or `configure` and a similar work on generic GitOps operations is provided. Then, a mapping from DT to generic GitOps operations is proposed. Finally, a proof-of-concept involving a Fischertechnik Training Factory 4.0 as a use-case is presented.

Keywords: Digital Twin · Model-Driven Engineering · DevOps· GitOps.

1 Introduction

Coined by Grieves [9], the term 'Digital Twins' (DTs) defines virtual clones of '*something*'. This 'something', or *Physical Object* (PO), can be anything ranging from an industrial supply chain to humans [7]. One of the most common ways to picture a DT is to decompose it in three parts: a PO, a Digital Object and

a Data Flow linking them together [16]. The pair of physical and digital objects is considered a DT if, and only if, the Data Flow between them is automated. Many alternative definitions, such as the 5D model by Tao et al., exist [7, 33]. DTs purpose is to maintain one or several models up-to-date by mirroring the PO. In a perfect scenario, they can also be used to carry out actions on the PO. This continuous synchronization between the DT and its physical system entails multiple (re)configurations, (re)connections and (re)deployments during the lifetime of the DT [1].

Many recent contributions focus on architectures and frameworks to improve the engineering (description, implementation, and federation) of DTs [11, 17, 19, 31]. This comprises not only academic works but also mature open-source frameworks such as Eclipse's Vortolang[3] and Microsoft's Digital Twin Definition Language (DTDL)[4] which are both popular options for describing DTs. While these solutions are suitable for the static description of DTs [27], they are not tailored to: *i)* the dynamic description of the aforementioned recurrent DT life cycle operations and, *ii)* the management of different configurations of DTs [22].

As DTs are model-centric [13], there is a growing interest in applying Model-Driven Engineering tools and techniques as a mean to deal with their sheer complexity, such as in their description and management [25]. However, in their recent systematic mapping study, Lehner et al. highlight that most MDE contributions primarily focused on the implementation life cycle of the DT, leaving a gap at the operational phase [19].

GitOps is an operational framework that combines Git version control with DevOps [2, 21] best practices to automate the deployment, scaling, and management of applications. The advantages of using Git as the operation center include: *i)* strong integration with a wide range of tools and services such as continuous integration and continuous deployment (CI/CD) pipelines, *ii)* version control and code histories, and *iii)* efficient handling of large, complex projects using branching strategies.

In order to improve the management of recurrent DTs life cycle operations, this paper proposes an approach to integrate DevOps practices into the life cycle management of DTs, leveraging MDE and GitOps. To achieve this goal, a definition of the DT life cycle operations is presented. Then, a vendor-independent conceptual model of GitOps is developed. Subsequently, an approach, combining elements from DTDL, GitOps and DT life cycle operations is presented. Using these elements as inputs, a process to generate GitOps artifacts that can be executed and manipulated is presented. Finally, a mapping between life cycle operations and GitOps elements is proposed.

The remainder of the paper is structured as follows. Section 2, gives an overview of DT modeling languages, the lacks in dynamic description of DTs of these languages and how MDE, through DTDL and GitOps, can be a solution in solving this issue. Section 3 presents DTs life cycle operations. Section 4 proposes a conceptual model of GitOps crafted as an abstraction of popular

[3] `https://eclipse.dev/vorto/`
[4] `https://github.com/Azure/opendigitaltwins-dtdl`

tools. Section 5 presents an approach to enable the systematic description of DTs life cycle operations by mapping them to GitOps elements. Section 6 gives a proof-of-concept implementation of the approach over a Fischertechnik Training Factory 4.0. Section 7 discusses related work and limitations of the approach. Section 8 concludes the paper.

2 Background

This section introduces DT modeling languages and more specifically the Digital Twin Definition Language. It also introduces GitOps.

2.1 Digital Twin modeling and DTDL

Many DT modeling languages have been proposed [7, 19, 27, 28, 32]. These languages have mostly been employed for the structural modeling of DTs. Among them, DTDL [12], defined by Microsoft, is a popular open-source standard for describing DTs of different kinds (assets, IoT systems, environments, etc.) in the form of JSON-LD files. DTDL models can be created by hand, or using a graphical user interface with the Azure Digital Twins (ADT) web-service. While it proposes functionalities which are common to other languages, it notably differentiates itself by providing the means to represent Relations and Composition between DTs [30]. A fully featured proprietary platform allows one to instantiate the models, connect the DTs to actual devices and telemetry.

DTDL is made up of a set of classes that are used to structurally describe DTs. The central class, `Interface`, which is used to describe any DT, is complemented by five classes enabling the description of telemetry data, properties, commands, relationships and components. Each DTDL element has an identifier, a Digital Twin Model Identifier (DTMI), in the form of `<scheme> : <path> ; <version>`. Also, DTDL uses Internationalized Resource Identifiers (IRIs) to refer to DTDL language elements (*e.g.*, names) and model-defined elements (*e.g.*, schemas). In DTDL, a distinction is made between `Component` and `Relationship`. While the former describes components that are part of the Interface, the latter describes links between DTs, thus enabling graphs of DTs. [27]. Figure 1 presents an excerpt of the DTDL version 4 metamodel, inspired from Jacoby and Usländer [12].

Although DTDL offers the possibility to finely describe any DT, its approach is mainly static and lacks dynamic elements to fully answer the continuous redeployment, reconnections and reconfiguration matter of the DT.

2.2 GitOps, a relative of DevOps

DevOps enables the continuous life cycle management of a software by improving the collaboration between development (Dev) and operational (Ops) teams. Over the years, it has proven to be successful in regular software applications and is now making its way to Cyber-Physical Systems (CPS) and more recently,

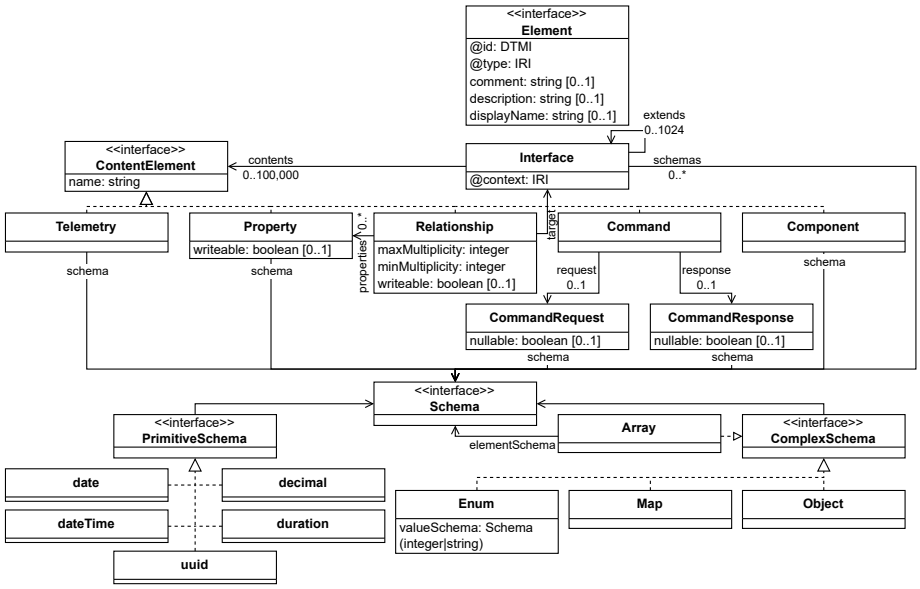

Fig. 1. Digital Twin Definition Language version 4 metamodel (excerpt), inspired from Jacoby et al. [12]

DTs [11,15]. It enables a better management of complex systems (*e.g.*, DT) and better integration of different components of such systems.

GitOps is a software development and deployment strategy centered around Git, a version control system, as the *source of truth*. Its main goals are to provide better efficiency, reliability and traceability compared to traditional software development strategies. These goals can be achieved thanks to a declarative description of the infrastructure and a process to make the environment match the declared infrastructure, mainly through orchestrators [34]. In a few words, GitOps is a way of implementing CI/CD pipelines for applications. The benefits of using GitOps include: faster deployment life cycles, easy declarative configuration, configuration drift detection, fast error recovery, and better stability and reliability [20,34].

3 Digital Twin Life Cycle

This section describes the DT life cycle at runtime and a running example. First, *core* DT life cycle operations and a simple DT life cycle are proposed. Then, a running example featuring the `connect` operation is presented to illustrate the approach. This operation, specific to the DT, involves not only a software connection to endpoints but also a physical connection to the PO and the sensors.

3.1 DT life cycle operations

This work focuses on the *Operate* phase of the DevOps strategy [24] by establishing more tailored DT-specific operations. Research on the merging of DTs and DevOps exhibits plenty of operations [25]. For this paper, operations are categorized into two categories: *i)* operations related to the PO, *e.g.*, the replacement of a machine in a factory and, *ii)* operations related to the DT as a software application, *e.g.*, the deployment of the DT to a platform. Operations of type *ii)* are further refined into two subcategories: tool-specific operations and general operations. Our approach focuses on general operations related to the life cycle of the DT.

If an operational problem happens, it is unlikely that the DT will provide accurate and up-to-date information to the user, removing the benefits of using a DT. Thus, a smooth continuity of these operations contributes to better quality of services proposed by the DT [19, 25]. Thanks to both the literature and experiments feedback when deploying an in-house DT platform, a catalog of general operations is proposed.

Each of these operations is considered atomic, meaning that it cannot be further decomposed, and is applied to a single, atomic DT. Thanks to the composition capabilities in DTDL, if an operation is applied to a composite DT, we expect this operation to propagate to each DT composing the main DT. In the best case scenario, the operation should only redeploy the modified atomic DT.

These operations compose *DT 'core' life cycle management operations* and form a basis for our approach. A representation of the succession of these operations as a state-machine diagram is depicted in Figure 2.

In this diagram, each life cycle operation appears as a transition and each target state is named `{operation}-ed`. The only exceptions are the In-Sync and Out-of-Sync states which are inside the `Connected` state. This is because the DT can transition from being In-Sync or Out-of-Sync when the connection is established to the PO [25]. While the `synchronize` operation allows to enter the In-Sync state, the transition to Out-of-Sync happens through *events* such as an evolution of the PO or a drift from the DT.

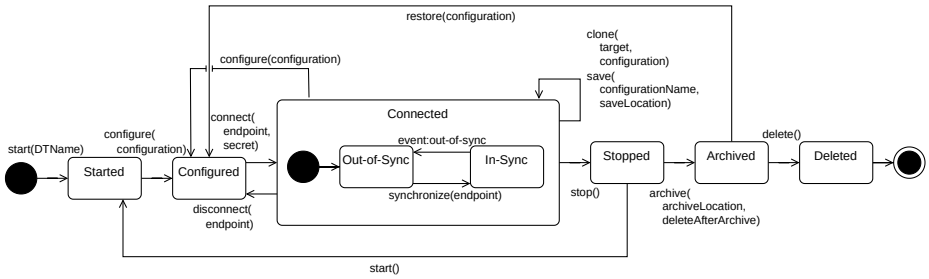

Fig. 2. Simple DT life cycle as a state-machine

A detailed definition of each operation is given below:

- `start(DTName: String)` scaffolds the DT [31]. None of the parameters or variables of the DT are set yet. The `DTName` permits to identify the DT. This operation always happens first. At this stage, the only possible action is to `configure` the DT.
- `configure(configuration: Configuration)` takes the DT provided by the `start` operation and configures specific variables of the DT [1,13,25,31]. A `Configuration`, which is a specific state of the DT (*e.g.*, parameters, tools, physical structure) at a given time, provides the necessary information to be initialized. This operation is available for the first time after a `start` operation and may be triggered at any time while the DT is connected, causing a reconfiguration. If the DT is in a twinning phase (see below), it will disconnect. The next possible operation is `connect`.
- `connect(endpoint: URI, secret: Secret (optional))` connects the DT to the PO by means of an endpoint, represented by a URI, and authentication secrets if necessary [31]. It connects the DT to a particular sensor or component of the PO. Once the connection is established, the DT enters a connected state in which the *twinning*, *i.e.*, the process of maintaining the representation of the PO up-to-date, happens. A `connect` operation usually happens after a `configure` operation since the latter automatically disconnects the DT when applied. The next possible operations are `save`, `clone`, `synchronize` and `stop`.
- `disconnect(endpoint: Endpoint (optional))`: disconnects an endpoint (if specified) or the entire DT from the PO [31]. Once applied, the DT no longer receives real-time data from the PO, thus the *twinning* process stops. This operation only applies if the DT is connected. If not, nothing happens and / or an error can be raised. Afterward, it is possible to `connect` again to the PO.
- `synchronize(endpoint: Endpoint (optional))`: applies in case of a desynchronization with the PO (the latest available data can not be pulled by the DT) [25, 31]. This operation only happens if at least one connection to a sensor (or the whole PO) is established. A synchronize operation can be triggered to force a reconnection. At this stage, the possible operations are: `clone`, `configure`, `save`, and `stop`.
- `clone(target: DT, configuration: Configuration (default: latest))`: clones this DT to a new target DT. More specifically, it copies all the data and models into a new target DT based on a given configuration. It clones the latest configuration by default. This operation is applied as long as a configuration of the DT exists. What's more, it does not change the state of the current DT. The next possible operations depend on the current state of the DT.
- `save(configurationName: string, saveLocation: string)`: saves the current configuration of the DT [31] using `configurationName` as the name of the configuration and `saveLocation` as a save path. This operation does not change the state of the DT and can be applied as long

as the DT has been configured at least once and is not deleted. The next available operations depend on the current state of the DT.

- `stop()` stops the DT, thus disconnecting from the PO and de-configuring the DT [13, 31]. From now on, new knowledge (data, knowledge) can be added to the DT but not in real-time. This state can serve maintenance or upgrade purposes. This operation can be applied if the DT is configured and connected to the PO. A restart of the DT is possible thanks to the `start` operation.
- `archive(archiveLocation: string, deleteAfterArchive: boolean (optional))` puts the repository containing the data of the DT into a read-only mode [25]. No further knowledge (models, data, configurations) can be modified after this [25]. A `deleteAfterArchive` option allows to immediately `delete` the DT after it has been archived. `archive` can only happen after a `stop` operation. The possible operations after this are `restore` and `delete`.
- `restore(configuration: Configuration (default: latest))` undoes the `archive` operation, putting the DT repository into read-write mode [25]. Connections and configurations of the DT are re-enabled. This operation necessitates to specify a version to restore to. By default, it uses the latest known configuration. Possible operation after this one is `connect`.
- `delete()` deletes all the data related to the DT (repository, data, models, connections...) [13, 25]. It can only happen after the DT has been archived and is irreversible. No further operation can be applied after this one.

3.2 Running example

An example of the `connect` operation is given below using a Fischertechnik Factory 4.0. This training model of a real-world factory, helps us illustrating how GitOps and DTDL could facilitate the connection to the PO. This example is based on the Sorting Line based on color Detection (SLD) module of the factory. An instantiation of this module can be found in Figure 3.

The traditional way of connecting the SLD module to a telemetry tool (in this case MQTT Broker) is to *hand-write* scripts using the information provided by the DTDL model. This task can be tedious, requiring to go back and forth between the script and the model. Furthermore, if a change happens physically or within the DTDL model, the coder needs to manually change the script, which is not instant and can lead to errors.

However, a direct integration of DTDL elements within GitOps could help us alleviate such risks. Indeed, when the DT of the module performs the `connect` operation, if a process could automatically retrieve the information from the DTDL model, then the time needed to update the information on the running would be much lower. This could also reduce the risks of typing errors.

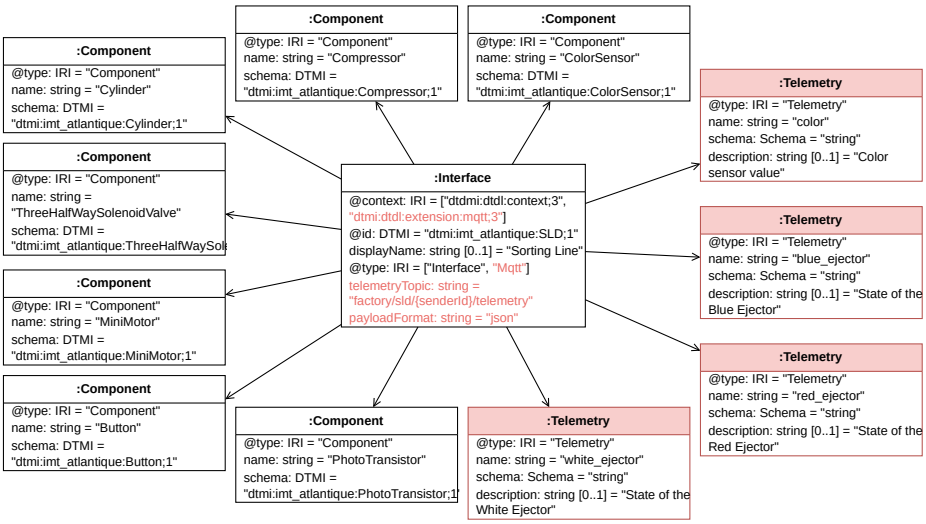

Fig. 3. Instantiation of the Sorting Line based on color Detection (SLD) module of the Fischertechnik Factory 4.0. Connection-related elements are represented in red.

4 GitOps conceptual modeling

This section covers the conceptual modeling of GitOps. First, we extract GitOps concepts by analyzing popular tools. Then, we present a conceptual model, leveraging the concepts we extracted.

4.1 Extraction of GitOps concepts

This paper proposes to establish a direct link between *a)* DT life cycle operations and *b)* GitOps elements. Thus, a unified model is required to achieve abstraction across different tools. As GitOps definition is not formal [34], we propose to extract a conceptual model by analyzing GitOps tools: ArgoCD[5], Flux[6] and Jenkins X[7]. These tools are regarded as the most common and popular tools to setup GitOps environments [10].

Table 1 gathers common concepts observed in the different tools. The main element is the *GitOps platform* as it is what we are conceptualizing. A simple definition of a GitOps platform can be: tool set that automates infrastructure and software application deployment by using Git as the source-of-truth. This means that in case of conflicts or errors, Git is the source of what we trust. The two main components of Git are *Git repositories* and *Git operations*. The former refers to the hosts that store the software applications, while the latter refers to the operations performed on those hosts.

[5] `https://argo-cd.readthedocs.io/`
[6] `https://fluxcd.io/`
[7] `https://jenkins-x.io/`

A common element of GitOps is *cloud providers*. These represent companies providing cloud computing services through different *tools*. There are many tools that support GitOps workflows today. Another component of cloud platforms are *artifact registries*. These hosts different kinds of *artifacts* produced by tools, ranging from *container images* to *packages*, *CI/CD pipelines*, etc.

This analysis of GitOps tools helped us identify other types of artifacts. First, we have *app configuration artifacts* which define the desired state of an application. If the application deviates from the desired state, a GitOps tool picks up this information and applies fixes to remediate the situation. Then, we have *container image recipes* consisting in a series of steps to define how to build container images. These steps may contain tool-specific operations or terminal commands.

Table 1. GitOps concepts

Concept	**Description**
GitOps platform	This represents the GitOps platform we are modeling
Git repository	The main element of GitOps which hosts software files.
Git operations	Set of operations which can be performed on the Git repository.
Cloud provider	Company that deliver cloud computing services. These services are provided by tools hosted on the platform.
Tool	Software application which provides a service. They can be provided by cloud providers or standalone (self-hosted).
Artifact registry	Centralized repository which hosts artifacts of different kinds (container images, packages, CD/CI pipelines, etc.)
CI/CD Pipelines	Series of steps which should deliver new software. Usually stored in JSON or YAML files.
App configurations	Define the configuration of a software application.
Orchestrator manifest	Piece of code which declares the desired state of a cluster.
Container Image Recipe	Series of steps which define how to build a new container image.
Container Image	Lightweight, standalone and executable software package which contains everything needed to run an application.

4.2 GitOps conceptual model

Figure 4 gathers the concepts identified in Section 4.1 as a GitOps conceptual model. The center piece of the figure is the GitOps platform. A platform has a name and an optional description. First, one or many *Git Repositories* compose the platform. Each repository has a name and can have a description. A remote URL locates the repository on the network. Different branches, one of which is the default branch, compose Git repositories. Visibility can be set using keywords

as *Public* (everyone can access), *Internal* (limited to some groups of users) or *Private* (only the owner and designated users can access).

Another main component of the Git tool is *Git Operations*. Identified main Git operations are *Commit* to locally save changes to the code, *Push* to send one or many commits to the Git repository, *Fetch* to check for changes on the repository, *Pull* to retrieve changes pushed to the repository, *Diff* to show changes between two versions of the code and *Merge* to join multiple development histories together.

A GitOps platform can also be linked to one or many *Cloud Providers*. Tools, which can be provided by a cloud provider or as standalone, self-hosted applications are also part of the GitOps platform. Different kinds of tools are identified, an *Orchestrator* orchestrates application clusters, an *Operator* operates the different GitOps operations, a *SyncTool* detects changes in the code, a *Database* stores data and a *DiffTool* detects the differences between the desired state and the actual state of the application.

CI/CD Pipelines play a key role within GitOps platforms. These pipelines have a name and are stored as files at given file locations. Different *Parameters* and a set of *Stages*, or jobs, to produce different artifacts compose these pipelines. At least one *Step* composes a job.

Finally, an *Artifact Registry* is identified by a name, an URL and a type to restrict the type of artifacts they can host. An *Artifact* has a name, a URI to access the artifact and it can have a description. Different types of artifacts are identified. *Orchestrator Manifests* which are composed of at least one *Orchestrator Object* and *Container Image Recipes* which can build *Container Images* artifacts based on a set of *Image Recipe Instructions*.

5 GitOps artifacts generation process

This section focuses on the description of the process to generate tangible GitOps artifacts and the mapping between previously described DT operations and GitOps elements for the systematic life cycle management of DTs.

5.1 Approach description

This paper envisions a process which takes DTDL models, Git repositories containing the GitOps environment parameters, and the extracted GitOps model as inputs to generate GitOps artifacts tailored to the GitOps platform as outputs. To achieve such goal, the proposed process decomposes into multiple steps pictured in Figure 5.

First step. Involves that a DT Modeler produces DTDL models. The correct filling of the DTDL models is crucial to ensure that the generated artifacts will be usable. To craft the DTDL models, the DT modeler uses both the PO and the Git repositories containing the code to execute the DT. Once the models are finished, they get pushed to a Git repository. Good practices to produce DTDL

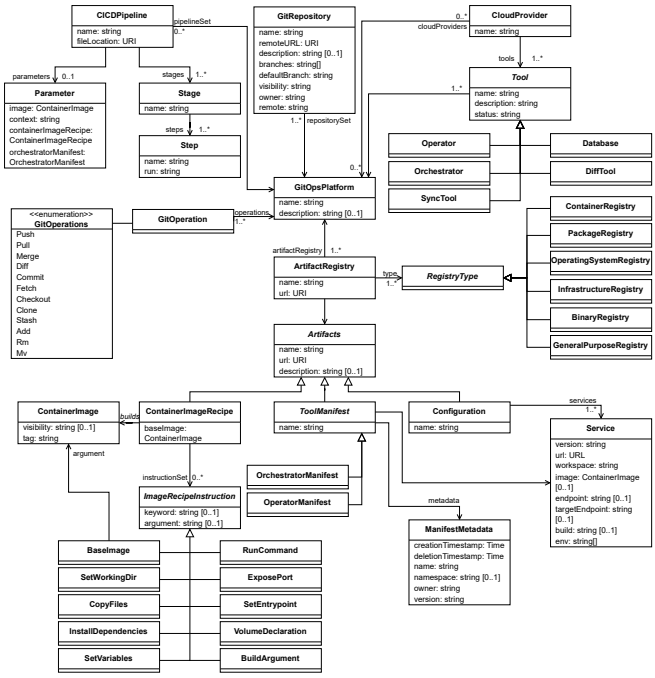

Fig. 4. Model of GitOps

models include the use of already existing industry-standard DTDL ontologies, the crafting of one interface per file and the validation of models.

Second step. Simultaneously or asynchronously, a DevOps/GitOps engineer defines a GitOps infrastructure for the DT. This involves to choose one or many GitOps tools. This produces artifacts such as infrastructure specification and tool configuration which are pushed to the GitOps repository.

Third step. The third step is an instantiation phase and necessitates the first and second steps to be complete. In this step, an actor takes the DTDL models and the set up GitOps platform to manually instantiate a GitOps model (presented in Section 4.2) using the mapping proposed below. For the `connect` operation, the actor fills the identified GitOps elements in the model using the `Telemetry` and `MQTT` DTDL elements and the tools `Configuration`.

Fourth step. Using the GitOps model instances crafted in the previous step, GitOps artifacts adapted to the platform get generated. These tailored artifacts, ranging from CI/CD pipelines to GitOps configuration files and manifests, can be pushed to the GitOps platform in order to be executed.

Ideally, this process does not regenerate every artifacts each time an update is made to the DT or the GitOps platform. Only the updated parts are regenerated, *e.g.*, for a connect operation, if an endpoint changes, only the parts related to that endpoint are regenerated. This should speed up the generation process and the deployment of new changes on the DT.

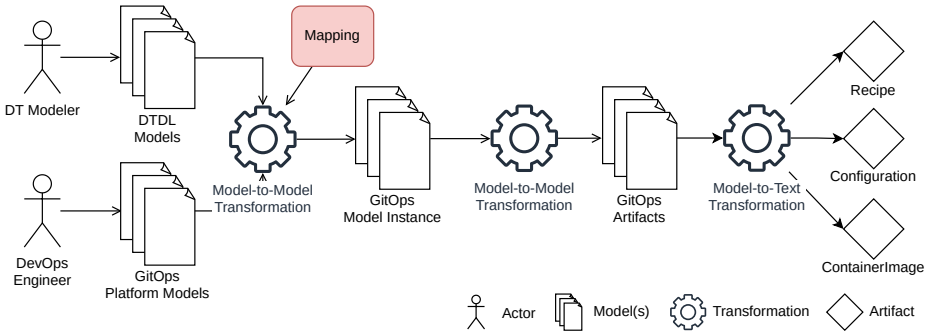

Fig. 5. Generation process from DTDL models and a GitOps configuration to platform-specific artifacts

5.2 DTDL and GitOps to Life Cycle Mapping

We establish a mapping between DT life cycle operations and DTDL and GitOps elements. This mapping is done manually, from experiments feedback.

The mapping is represented in Table 2. Each row shows which DTDL and GitOps elements can be associated with a given life cycle operation. While DTDL elements act as source of information for the generation process, GitOps elements of this mapping include both input elements and outputs elements. The mapping is not one-to-one, multiple elements from one set may correspond to multiple elements in the other.

As an example, in the `connect` operation, `Telemetry` and `MQTT` elements both contribute to set information about the endpoint we want to connect to. For GitOps elements, the `Git Repository` provides missing information in DTDL models such as tool-specification, workspaces and remote URLs.

6 Proof of concept: Fischertechnik Factory use case

As a mean of demonstration of feasibility of the approach, this paper proposes a proof of concept involving the `connect` operation and a Fischertechnik Training Factory 4.0.

The Fischertechnik Factory 4.0 [29] is a small training factory designed to simulate modern industrial processes, focusing on the principles of Industry 4.0. It includes components like sensors, actuators, and control systems that allow users to build and program automated production lines.

For this use case, we use our running example, the `connect` operation which can be summarized to: setup an MQTT Broker and connect (subscribe) to the right topic. Then, our approach should enable us to: gather the necessary MQTT parameters from the DTDL model and gather information from the GitOps configuration to craft a GitOps instance of the proposed model. A handmade

Table 2. Mapping between DT life cycle operations, DTDL and GitOps

Operation	DTDL element	GitOps element
start	Interface	GitRepository, GitOperations (clone), Operator, Orchestrator, CICDPipeline, AppConfiguration
configure	Interface, Property, Component	Operator, Orchestrator, CICDPipeline, ContainerImage, ToolManifest, AppConfiguration
connect	Telemetry, MQTT	ContainerImageRecipe, ContainerImage, ToolManifest, AppConfiguration
disconnect	Telemetry, MQTT	ToolManifest, AppConfiguration
synchronize	Telemetry, MQTT	GitRepository, GitOperations (fetch, pull)
clone	Interface, Telemetry, Property, Relationship, Command, Component, MQTT	GitRepository, GitOperations (fetch, pull, clone, commit), ContainerImage, ToolManifest, AppConfiguration, Database
save	Interface, Telemetry, Property, Relationship, Command, Component, MQTT	GitRepository, GitOperations (add, commit, push), ToolManifest, Database, CICDPipeline
stop	Interface	GitRepository, GitOperations (mv, add, commit, push), Operator, Orchestrator, ToolManifest, AppConfiguration
archive	Interface	GitRepository, ToolManifest, CICDPipeline, ContainerImage
restore	Interface, Telemetry, Property, Relationship, Command, Component, MQTT	GitRepository, GitOperations (mv, add, commit, push), CICDPipeline, ToolManifest
delete	Interface	GitRepository, GitOperations (rm, commit, push)

deployment of the `connect` operation reveals that expected artifacts are an orchestrator manifest, a container image recipe, a container image and a CI/CD pipeline.

First step. A scoped-down, but sufficient for this paper, DTDL model of the factory is handcrafted. We start by using the physical structure of the factory, *i.e.*, we declare each factory module as a DT in separate JSON files and also declare a main 'Fischertechnik Factory' DT which is composed of each module. The physical description in DTDL ends by describing each component for each module. Then, we use the software components at our hand to refine the DTDL model by adding MQTT connections and annotations. The model then get instantiated using an EMF model of DTDL. The crafted DTDL models can be found on the dedicated repository[8].

Second step. Concurrently, we setup, by hand, a GitOps platform on a machine using the ArgoCD operator and the Kubernetes[9] orchestrator. This produces configuration files for both ArgoCD and Kubernetes. This GitOps platform is then used as input to fill a GitOps platform EMF model.

Third step. We then instantiate identified GitOps elements in the mapping. This means handcrafting a `GitRepository`, a `Configuration` and a `Container ImageRecipe` objects (blue in Figure 6). To create these elements, we use both DTDL elements, mostly MQTT elements within the `Interface` and `Telemetry` and GitOps elements extracted from configuration files.

Fourth step. The GitOps elements instantiated in the third step are used to generate artifacts useful to perform the `connect` operation using Acceleo Model-to-Text transformations. These artifacts include an `OrchestratorManifest`, an `OperatorManifest` and a `ContainerImage`.

Application of the approach used the elements depicted in Figure 6. To make sure that the generated artifacts make sense, we compared them to the ones we handcrafted and found that they correspond closely syntactically and semantically.

7 Related Work & Discussion

In recent years, many different approaches to model DTs have been proposed in the literature [3,19,23,26,27]. We center our discussion on contributions specially focused on the modeling and management of DTs life cycle.

Kamburjan et al., describe an automatic method for self-adaptation on DTs to address shifts between life cycle phases in a PO [13]. They propose declarative life cycle stages expressed by using the Manchester OWN Syntax. Our work targets the DTs operational life cycle and thus may be seen as complementary to theirs. More similar to our work, Lehner et al. [18] propose a reference architecture to take advantage of the principles of MDE to connect models crafted at design times with running systems [18]. It proposes to use model repositories as artifacts from the design phase and to connect them to running systems and use

[8] `https://github.com/labsticc-p4s/er2025-proof_of_concept`
[9] `https://kubernetes.io/`

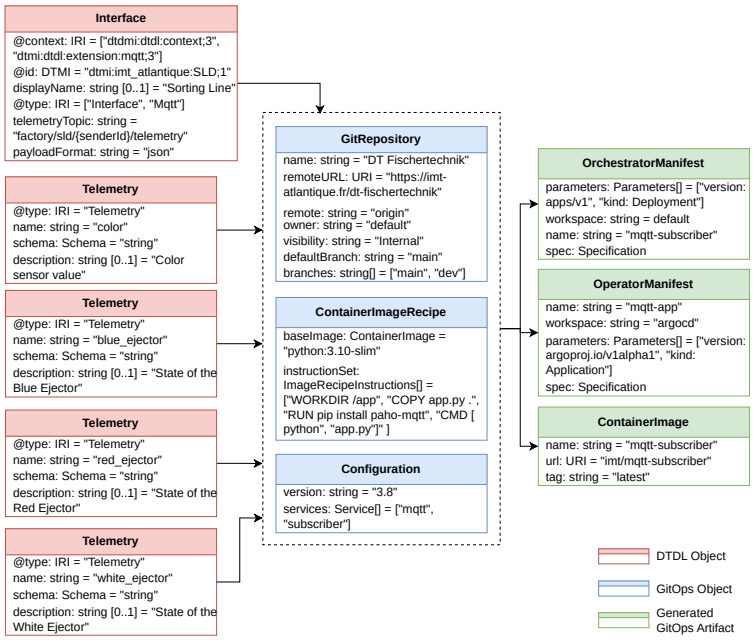

Fig. 6. Implementation of the *connect* operation on the SLD module of the Fischertechnik Factory 4.0 (simplified).

them as a source of information. Our work differentiates from Lehner's proposed architecture, by focusing on automating DTs life cycle operations by generating and orchestrating GitOps artifacts.

DevOps and life cycle management have also been studied from the Cyber Physical Systems perspective [5, 8, 18]. Koren et al. [14] showcase the DevOps4CPPS framework, spotlighting the use of microservices and GitOps in a CPPS environment. Furthermore, Aissat et al. [1] introduce JuNo-OPS, a DevOps framework establishing guidelines on using DevOps for DTs. Based on their experiment, they showcase the difficulties of collaboration between domain and non-domain experts (*e.g.*, git training). Our work takes inspiration from these efforts, but focuses on the particularities of the DT life cycle.

Recent works from Talasila et al. [31] and Dalibor et al. [6] center their attention on platforms for running DTs. Their platforms allow users to describe DTs life cycle operations through scripts. Our approach aims at the automatic generation and management of such scripts leveraging GitOps.

Finally, and focused on modeling the DevOps process and platform, Colantoni et al. [4] propose a framework, Dev-Ops-ML, which allows users to model DevOps platforms and engineering processes, but do not deal directly with DTs nor GitOps.

Thus, while this paper capitalizes on previous work, it differentiates itself by shifting the focus from the development part of DevOps to the operational

part. For instance, we have not seen solutions in the literature putting a clear focus on the life cycle operations of the DT at runtime. Generally, this part has been, at best, handled in an *ad hoc* manner. The idea of combining an existing DT modeling language with a DevOps operational framework like GitOps is original. We believe that the benefits of using a framework centered around Git and DevOps principles match the needs of DTs w.r.t. the management of their life cycle. While the initial setup and the general maintenance of a GitOps platform introduce complexity to the system, the automation of life cycle processes frees up useful time to engineers which they could allocate towards other parts of the DTs such as services.

8 Conclusion & Future Work

In this paper, we have presented a model-driven approach for the life cycle management of DTs. Our approach is based on the combination of a DT description language and GitOps. It includes the definition of a novel GitOps metamodel and the corresponding mapping between a DT description, a number of high level DT life cycle operations and GitOps operations and artifacts. A proof of concept demonstrating the feasibility of the approach has been presented as well.

In future work, we plan to both extend our approach to include additional operations and to work on the granularity of the ones presented here. Indeed the operations presented here refer to the DT as a whole, and not to parts of it (*e.g.*, we may want to disconnect just some part of a given DT). Furthermore, we intend to study DT configuration management, *i.e.*, manage different versions of a DT using Git branching strategies. Besides, we plan to study a DTDL extension (*e.g.*, by including life cycle operation annotations) which may facilitate the mapping of language elements in our approach. Additionally, we intend to extend the evaluation to larger and more complex DTs in order to evaluate the scalability and generalizability of our approach.

Finally, while Figure 2 proposes baseline DT operations, further investigations will focus on the customization and modularity of this approach to further align with DevOps principles.

Acknowledgments. Gwendal Beaumont's Ph.D. is funded by the Futur, ruptures & impacts program by Fondation Mines-Télécom and the TSN CARNOT institute. Authors thank Région Bretagne for the CPER Industrie du Futur 2021-2027 grant, and the ANR (French National Research Agency) MODES JCJC project n° ANR-23-CE25-0011-01 as well for partially funding this research.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aissat, S., Beaulieu, J., Bordeleau, F., Gascon-Samson, J., Poirier, E., Motamedi, A.: JuNo-OPS: A DevOps Framework for the Engineering of Digital Twins for Built

Assets. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. pp. 496–506. MODELS Companion '24, Association for Computing Machinery, Linz, Austria (2024). https://doi.org/10.1145/3652620.3688266, https://doi.org/10.1145/3652620.3688266

2. Beetz, F., Harrer, S.: GitOps: The Evolution of DevOps? IEEE Software **39**(4), 70–75 (Jul 2022). https://doi.org/10.1109/MS.2021.3119106, https://ieeexplore.ieee.org/document/9565152/?arnumber=9565152&tag=1, conference Name: IEEE Software
3. Bordeleau, F., Combemale, B., Eramo, R., Brand, M.v.d., Wimmer, M.: Towards Model-Driven Digital Twin Engineering: Current Opportunities and Future Challenges. In: Systems Modelling and Management. pp. 43–54. Springer International Publishing (Jun 2020). https://doi.org/10.1007/978-3-030-58167-1_4, https://doi.org/10.1007/978-3-030-58167-1_4
4. Colantoni, A., Berardinelli, L., Wimmer, M.: DevOpsML: towards modeling DevOps processes and platforms. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings. pp. 1–10. MODELS '20, Association for Computing Machinery, New York, NY, USA (Oct 2020). https://doi.org/10.1145/3417990.3420203, https://dl.acm.org/doi/10.1145/3417990.3420203
5. Combemale, B., Wimmer, M.: Towards a Model-Based DevOps for Cyber-Physical Systems. In: Bruel, J.M., Mazzara, M., Meyer, B. (eds.) Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment. pp. 84–94. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-39306-9_6
6. Dalibor, M., Heithoff, M., Michael, J., Netz, L., Pfeiffer, J., Rumpe, B., Varga, S., Wortmann, A.: Generating customized low-code development platforms for digital twins. Journal of Computer Languages **70**, 101117 (Jun 2022). https://doi.org/10.1016/j.cola.2022.101117, https://www.sciencedirect.com/science/article/pii/S2590118422000235
7. Dalibor, M., Jansen, N., Rumpe, B., Schmalzing, D., Wachtmeister, L., Wimmer, M., Wortmann, A.: A Cross-Domain Systematic Mapping Study on Software Engineering for Digital Twins. Journal of Systems and Software **193**, 111361 (Nov 2022). https://doi.org/10.1016/j.jss.2022.111361, https://linkinghub.elsevier.com/retrieve/pii/S0164121222000917
8. Dobaj, J., Riel, A., Macher, G., Egretzberger, M.: Towards DevOps for Cyber-Physical Systems (CPSs): Resilient Self-Adaptive Software for Sustainable Human-Centric Smart CPS Facilitated by Digital Twins. Machines **11**(10), 973 (Oct 2023). https://doi.org/10.3390/machines11100973, https://www.mdpi.com/2075-1702/11/10/973, number: 10 Publisher: Multidisciplinary Digital Publishing Institute
9. Grieves, M.: Product lifecycle management: driving the next generation of lean thinking. McGraw-Hill, New York (2006)
10. Gupta, S., Bhatia, M., Memoria, M., Manani, P.: Prevalence of GitOps, DevOps in Fast CI/CD Cycles. In: 2022 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COM-IT-CON). vol. 1, pp. 589–596 (May 2022). https://doi.org/10.1109/COM-IT-CON54601.2022.9850786, https://ieeexplore.ieee.org/abstract/document/9850786
11. Hugues, J., Hristosov, A., Hudak, J.J., Yankel, J.: TwinOps - DevOps meets model-based engineering and digital twins for the engineering of CPS. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Lan-

guages and Systems: Companion Proceedings. pp. 1–5. MODELS '20, Association for Computing Machinery, New York, NY, USA (Oct 2020). https://doi.org/10.1145/3417990.3421446, https://dl.acm.org/doi/10.1145/3417990.3421446

12. Jacoby, M., Usländer, T.: Digital Twin and Internet of Things—Current Standards Landscape. Applied Sciences **10**(18), 6519 (Jan 2020). https://doi.org/10.3390/app10186519, https://www.mdpi.com/2076-3417/10/18/6519, number: 18 Publisher: Multidisciplinary Digital Publishing Institute
13. Kamburjan, E., Bencomo, N., Tarifa, S.L.T., Johnsen, E.B.: Declarative Lifecycle Management in Digital Twins. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. pp. 353–363. MODELS Companion '24, Association for Computing Machinery, Linz, Austria (2024). https://doi.org/10.1145/3652620.3688248, https://doi.org/10.1145/3652620.3688248
14. Koren, I., Rinker, F., Meixner, K., Kröger, M., Zeng, M.: Implementing DevOps Practices in CPPS Using Microservices and GitOps. In: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA). pp. 1–4 (Sep 2023). https://doi.org/10.1109/ETFA54631.2023.10275433, https://ieeexplore.ieee.org/document/10275433, iSSN: 1946-0759
15. Koren, I., Rinker, F., Meixner, K., Matevska, J., Walter, J.: Challenges and Opportunities of DevOps in Cyber-Physical Production Systems Engineering. In: 2023 IEEE 6th International Conference on Industrial Cyber-Physical Systems (ICPS). pp. 1–6 (May 2023). https://doi.org/10.1109/ICPS58381.2023.10128073, https://ieeexplore.ieee.org/document/10128073/?arnumber=10128073&tag=1, iSSN: 2769-3899
16. Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W.: Digital Twin in manufacturing: A categorical literature review and classification. IFAC-PapersOnLine **51**(11), 1016–1022 (Jan 2018). https://doi.org/10.1016/j.ifacol.2018.08.474, https://www.sciencedirect.com/science/article/pii/S2405896318316021
17. Lee, A., Manrique Negrin, D.A., Cleophas, L.: Evaluating Open-Source Tools for Heterogeneous Model-Based Digital Twin Development: A Microbrewery Case Study. In: 2024 Annual Modeling and Simulation Conference (ANNSIM). pp. 1–13 (May 2024). https://doi.org/10.23919/ANNSIM61499.2024.10732658, https://ieeexplore.ieee.org/document/10732658/?arnumber=10732658&tag=1
18. Lehner, D., Wolny, S., Mazak-Huemer, A., Wimmer, M.: Towards a Reference Architecture for Leveraging Model Repositories for Digital Twins. In: 2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). vol. 1, pp. 1077–1080 (Sep 2020). https://doi.org/10.1109/ETFA46521.2020.9212109, https://ieeexplore.ieee.org/document/9212109/, iSSN: 1946-0759
19. Lehner, D., Zhang, J., Pfeiffer, J., Sint, S., Splettstößer, A.K., Wimmer, M., Wortmann, A.: Model-driven engineering for digital twins: a systematic mapping study. Software and Systems Modeling (Mar 2025). https://doi.org/10.1007/s10270-025-01264-7, https://doi.org/10.1007/s10270-025-01264-7
20. Limoncelli, T.A.: GitOps: A Path to More Self-service IT: IaC + PR = GitOps. Queue **16**(3), 13–26 (Jun 2018). https://doi.org/10.1145/3236386.3237207, https://dl.acm.org/doi/10.1145/3236386.3237207
21. Linsbauer, S., Hark, R., Koziolek, H., Eskandani, N.: Runtime Orchestration of Distributed Control System Services with TOSCA, Kubernetes, and GitOps. In: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-

C). pp. 40–47 (Jun 2024). https://doi.org/10.1109/ICSA-C63560.2024.00013, https://ieeexplore.ieee.org/abstract/document/10628307, iSSN: 2768-4288

22. Lu, H., Yue, T., Ali, S.: Pattern-based Interactive Configuration Derivation for Cyber-physical System Product Lines. ACM Trans. Cyber-Phys. Syst. **4**(4), 44:1–44:24 (Jun 2020). https://doi.org/10.1145/3389397, https://dl.acm.org/doi/10.1145/3389397
23. Maree, J.M.J., Kruger, K., Basson, A.H.: A Digital Twin Architecture for the Provisioning, Management, and Monitoring of Heterogenous IoT Devices. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. pp. 442–452. MODELS Companion '24, Association for Computing Machinery, Linz, Austria (2024). https://doi.org/10.1145/3652620.3688258, https://doi.org/10.1145/3652620.3688258
24. Mejía Miranda, J., Muñoz, M., Rocha, A., Hernández Pérez, Y., Avila-George, H. (eds.): New Perspectives in Software Engineering. No. 1135 in Studies in Computational Intelligence, Springer Nature Switzerland, Cham, 1st ed. 2024 edn. (2024). https://doi.org/10.1007/978-3-031-50590-4
25. Michael, J., Cleophas, L., Zschaler, S., Clark, T., Combemale, B., Godfrey, T., Khelladi, D.E., Kulkarni, V., Lehner, D., Rumpe, B., Wimmer, M., Wortmann, A., Ali, S., Barn, B., Barosan, I., Bencomo, N., Bordeleau, F., Grossmann, G., Karsai, G., Kopp, O., Mitschang, B., Munoz Ariza, P., Pierantonio, A., Polack, F., Riebisch, M., Schlingloff, H., Stumptner, M., Vallecillo, A., van den Brand, M., Vangheluwe, H.: Model-Driven Engineering for Digital Twins: Opportunities and Challenges. Systems Engineering Journal (Mar 2025)
26. Negrin, D.A.M., Cleophas, L., Van Den Brand, M.: Using Ptolemy II as a Framework for Virtual Entity Integration and Orchestration in Digital Twins. In: 2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C). pp. 233–236 (Oct 2021). https://doi.org/10.1109/MODELS-C53483.2021.00041, https://ieeexplore.ieee.org/abstract/document/9643793
27. Pfeiffer, J., Lehner, D., Wortmann, A., Wimmer, M.: Modeling Capabilities of Digital Twin Platforms - Old Wine in New Bottles? The Journal of Object Technology **21**(3), 3:1 (2022). https://doi.org/10.5381/jot.2022.21.3.a10, http://www.jot.fm/contents/issue_2022_03/article10.html
28. Qi, Q., Tao, F., Hu, T., Anwer, N., Liu, A., Wei, Y., Wang, L., Nee, A.Y.C.: Enabling technologies and tools for digital twin. Journal of Manufacturing Systems **58**, 3–21 (Jan 2021). https://doi.org/10.1016/j.jmsy.2019.10.001, https://doi.org/10.1016/j.jmsy.2019.10.001
29. Sala, R., Pirola, F., Pezzotta, G.: On the development of the Digital Shadow of the Fischertechnik Training Factory Industry 4.0: an educational perspective. Procedia Computer Science **217**, 640–649 (2023). https://doi.org/10.1016/j.procs.2022.12.260, https://linkinghub.elsevier.com/retrieve/pii/S1877050922023389
30. Schmidt, C., Volz, F., Stojanovic, L., Sutschet, G.: Increasing Interoperability between Digital Twin Standards and Specifications: Transformation of DTDL to AAS. Sensors **23**(18), 7742 (Jan 2023). https://doi.org/10.3390/s23187742, https://www.mdpi.com/1424-8220/23/18/7742, number: 18 Publisher: Multidisciplinary Digital Publishing Institute
31. Talasila, P., Gomes, C., Mikkelsen, P.H., Arboleda, S.G., Kamburjan, E., Larsen, P.G.: Digital Twin as a Service (DTaaS): A Platform for Digital Twin Developers and Users. In: 2023 IEEE Smart World Congress (SWC). pp. 1–8 (Aug

2023). https://doi.org/10.1109/SWC57546.2023.10448890, http://arxiv.org/abs/2305.07244, arXiv:2305.07244 [cs]
32. Tao, F., Xiao, B., Qi, Q., Cheng, J., Ji, P.: Digital twin modeling. Journal of Manufacturing Systems **64**, 372–389 (Jul 2022). https://doi.org/10.1016/j.jmsy.2022.06.015, https://www.sciencedirect.com/science/article/pii/S0278612522001108
33. Tao, F., Zhang, H., Liu, A., Nee, A.Y.C.: Digital Twin in Industry: State-of-the-Art. IEEE Transactions on Industrial Informatics **15**(4), 2405–2415 (Apr 2019). https://doi.org/10.1109/TII.2018.2873186, https://ieeexplore.ieee.org/document/8477101, conference Name: IEEE Transactions on Industrial Informatics
34. Weaveworks: GitOps — Operations by Pull Request (Sep 2018), https://medium.com/weaveworks/gitops-operations-by-pull-request-14e8b659b058